Technical Mastery Guide

The Mechanics and Mathematical Foundations of Logistic Regression

A Structured Reference for Binary Classification, the Sigmoid Function, Decision Boundaries, and the Logistic Cost Function

- 1. Binary Classification
- 2. Limits of Linear Regression
- 3. Sigmoid & the Logistic Model
- 4. Decision Boundaries
- 5. Cost Function & Convexity
- 6. Implementation Summary

1. Introduction to Binary Classification

Binary classification is a fundamental supervised learning task where the objective is to predict an output variable y that can only take on one of two discrete values: 0 or 1. Unlike linear regression, which predicts continuous numerical values across an infinite range, classification categorizes data into specific, distinct classes.

Standard Terminology

In binary classification, the two possible classes are referred to using the following standard terms:

- **The Negative Class**: Represented by 0, “False”, or “No”. This denotes the absence of a condition, such as a benign tumor, a non-fraudulent transaction, or a legitimate email.
- **The Positive Class**: Represented by 1, “True”, or “Yes”. This denotes the presence of the condition being studied, such as a malignant tumor, a fraudulent transaction, or a spam email.

It is critical to understand that “positive” and “negative” are technical labels used to convey the presence or absence of a feature; they do not imply a value judgment of “good” versus “bad.”

A Note on Nomenclature

One common point of confusion for students is the name “Logistic Regression.” Despite the word “regression” appearing in its title, this algorithm is used exclusively for classification. The name exists for historical reasons, but in modern machine learning, it remains one of the most popular and effective tools for solving binary classification problems.

2. The Limitations of Linear Regression in Classification

While linear regression is a powerful tool for predicting continuous values, it fails as a robust classification algorithm due to its mathematical and geometric properties.

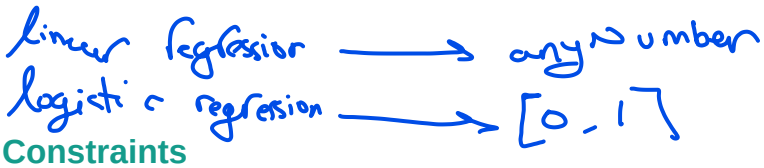
The Thresholding Problem

A naive approach to classification involves fitting a linear regression line to the data and applying a threshold, typically at 0.5. In this setup, if the model outputs a value ≥ 0.5, we predict $y = 1$; if it outputs < 0.5, we predict $y = 0$.

The Impact of Outliers and Slope Mechanics

The primary failure of linear regression in this context is its extreme sensitivity to outliers. Consider a “Tumor Size” dataset where most data points are clustered near the origin. If a single training example of a very large tumor is added far to the right, the linear regression algorithm attempts to minimize the squared error for all points. To accommodate this outlier, the slope of the best-fit line becomes significantly shallower.

This change in slope pushes the 0.5 threshold intersection point further to the right. Consequently, the decision boundary shifts, causing the model to misclassify existing positive examples as negative simply because a single extreme example was introduced.



Range Constraints

Linear regression is unbounded. It can produce predictions significantly less than 0 or greater than 1. For classification, we require a model bounded within the interval [0, 1] to allow for a valid probabilistic interpretation.

3. The Sigmoid Function and the Logistic Model

To overcome these limitations, we utilize the Sigmoid function (also known as the Logistic function) to map any real-valued number into a range between 0 and 1.



Mathematical Definition

The Sigmoid function g(z) is defined as:

FORMULA — SIGMOID FUNCTION

$$g(z) = 1 / (1 + e^{-z})$$

where e is the mathematical constant approximately equal to 2.718.

Asymptotic Behavior

The S-shaped curve of the Sigmoid function exhibits specific behaviors across the range of z:

Condition	Behavior of e^{-z}	Result: g(z)
As $z \to \infty$	approaches 0	$g(z) \to 1$
As $z \to -\infty$	grows toward infinity	$g(z) \to 0$
When z = 0	$e^0 = 1$	g(z) = 0.5

Model Composition

The logistic regression model $f_{w,b}(x)$ is constructed by passing a linear expression z through the Sigmoid function. Given a feature vector x, a weight vector w, and a bias b, we define:

FORMULA — MODEL COMPOSITION

$$z = w \cdot x + b$$

$$f_{w,b}(x) = g(z) = 1 / (1 + e^{-(w \cdot x + b)})$$

Probabilistic Interpretation

The output of the model is interpreted as the probability that the label y is 1, given the input x and parameters w, b. This is formally denoted as:

FORMULA — PROBABILISTIC INTERPRETATION

$$f_{w,b}(x) = P(y = 1 \mid x; w, b)$$

For example, if $f_{w,b}(x) = 0.7$, the model is predicting a 70% probability that the example belongs to the positive class (1) and a 30% probability that it belongs to the negative class (0), as the probabilities for both possible classes must sum to 1.

4. Decision Boundaries: From Linear to Non-Linear

The decision boundary is the specific line, curve, or manifold on a feature plot that separates the region where the model predicts y = 1 from the region where it predicts y = 0.

The Prediction Rule

Using the standard threshold of 0.5:

- Predict y = 1 if $f_{w,b}(x) \geq 0.5$. This occurs whenever $z \geq 0$ (i.e., $w \cdot x + b \geq 0$).
- Predict y = 0 if $f_{w,b}(x) < 0.5$. This occurs whenever $z < 0$ (i.e., $w \cdot x + b < 0$).

Geometry of the Boundary

It is vital to distinguish between the model function and the decision boundary. The model function f(x) is an S-shaped surface that maps features to probabilities. In contrast, the decision boundary is a flat representation on the feature space where z = 0.

When we use a single-order polynomial (e.g., $z = w_1x_1 + w_2x_2 + b$), the decision boundary is a straight line. However, by incorporating higher-order polynomial features—such as x_1^2 or x_2^2—the model can fit complex shapes. For instance, the expression below defines a circular decision boundary:

FORMULA — CIRCULAR DECISION BOUNDARY

$$z = x_1^2 + x_2^2 - 1$$

Points outside the circle where $z \geq 0$ are classified as 1, while points inside where $z < 0$ are classified as 0.

5. The Logistic Cost Function and Convexity

The cost function J(w, b) provides a mathematical measure of how well the model's parameters fit the training data.

The Failure of Squared Error

In linear regression, we use the squared error cost function. If we apply squared error to the non-linear Sigmoid function, the resulting cost surface is non-convex. This "wiggly" surface contains multiple local minima, which can trap gradient descent and prevent it from reaching the global minimum.

The Logarithmic Loss Function

To ensure a convex cost surface, we use a logarithmic loss function for individual examples:

True Label	Loss($f_{w,b}(x)$, y)
If y = 1	$-\log(f_{w,b}(x))$
If y = 0	$-\log(1 - f_{w,b}(x))$

This ensures that the loss approaches infinity as the prediction moves further from the true label (e.g., predicting a probability near 0 when the truth is 1). These two cases can be synthesized into a single simplified loss equation:

FORMULA — SIMPLIFIED LOSS

$$L(f_{w,b}(x^{(i)}), y^{(i)}) = -y^{(i)} \cdot \log(f_{w,b}(x^{(i)})) - (1 - y^{(i)}) \cdot \log(1 - f_{w,b}(x^{(i)}))$$

The Full Cost Function and Statistical Foundation

The total cost J(w, b) is the average loss over the entire training set of m examples:

FORMULA — TOTAL COST

$$J(w, b) = (1/m) \cdot \sum_{i=1}^{m} L(f_{w,b}(x^{(i)}), y^{(i)})$$

This specific cost function is derived from the statistical principle of Maximum Likelihood Estimation (MLE). MLE is a method for finding the parameters that maximize the probability of the observed data occurring. By using this derivation, we are guaranteed a convex cost function, ensuring that optimization algorithms will reliably converge to the global minimum.

6. Summary of Technical Implementation

The complete workflow for implementing Logistic Regression is summarized as follows:

1. **Feature Mapping**: Map input features x to the intermediate variable z via the linear expression z = w · x + b.
2. **Sigmoid Transformation**: Pass z through the Sigmoid function g(z) to produce an output bounded between 0 and 1.
3. **Probabilistic Interpretation**: Interpret the output $f_{w,b}(x)$ as the probability P(y = 1 | x; w, b).
4. **Thresholding**: Apply a decision boundary threshold (typically 0.5) to categorize the probability into a discrete class (0 or 1).
5. **Parameter Optimization**: Use Gradient Descent to minimize the convex logarithmic cost function J(w, b), iteratively adjusting w and b to find the global minimum.

Takeaway: Logistic regression converts a linear score $z = w \cdot x + b$ into a bounded probability via the Sigmoid function, draws its decision boundary at $z = 0$, and is trained by minimizing a convex log-loss cost function derived from Maximum Likelihood Estimation — which is precisely what guarantees Gradient Descent can reliably reach the global minimum.